



Práctica #6:

Ramificación y poda

12/05/2026

Daniel Palenzuela Álvarez

Universidad de La Laguna - Ingeniería Informática

Curso 2025/2026

Diseño y Análisis de Algoritmos

3º año

Introducción

El Maximum Diversity Problem (MDP) consiste en seleccionar un subconjunto de tamaño m dentro de un conjunto de n elementos de forma que la suma de las distancias entre los elementos elegidos sea máxima.

En esta práctica se han implementado dos formas, con un algoritmo voraz como constructivo inicial y un algoritmo ramificación y poda, con dos estrategias de expansión de nodos, priorizando la menor cota superior y la mayor cota superior.

Las instancias utilizadas tienen distintas combinaciones de tamaño y dimensión, con $n = 15, 20, 30$ y $K = 2, 3$, evaluando los valores de $m = 2, 3, 4, 5$. Para cada instancia se ha medido el valor objetivo z , la solución obtenida S , el tiempo de CPU y, en el caso de ramificación y poda, el número de nodos generados.

Algoritmos

El algoritmo voraz se implementa en la clase `AlgoritmoVoraz`, heredera de `Algoritmo`. En cuanto a su funcionamiento, el programa primero calcula el centroide del conjunto completo, selecciona el elemento más alejado de ese centroide, después repite el proceso sobre el subconjunto ya construido hasta completar m elementos. En el código, esto se divide en tres partes, con `centroide()` que calcula el centro de gravedad de un conjunto de elementos, `masAlejadoDelCentro()` que elige el candidato más lejano al centroide y `resolver()` que coordina todo el proceso voraz y calcula el valor final de la solución. Este algoritmo se usa como método rápido de construcción inicial y también como solución de referencia para la ramificación y poda, porque proporciona una buena cota inferior desde el principio.

El algoritmo de ramificación y poda se implementa en la clase `AlgoritmoRamificacionPoda`, también derivada de `Algoritmo`. Cada nodo del árbol representa una solución parcial.

En el código, el nodo se modela con la estructura `Nodo`, que guarda la selección parcial, el siguiente índice candidato, el valor parcial acumulado y la cota superior estimada. El método `resolver()` construye el árbol de búsqueda, genera los hijos de cada nodo y elimina los que no pueden mejorar la mejor solución encontrada. Antes de empezar, calcula una solución inicial con el voraz para usarla como cota inferior. La función `calcularCotaSuperior()` estima el mejor valor posible que se podría conseguir a partir de un

nodo parcial, y si esa cota no supera la mejor solución actual, el nodo se poda y no se explora más.

Se han implementado dos estrategias de exploración en clases separadas, con *EstrategiaMenorCotaSuperior* y *EstrategiaMayorCotaSuperior*. Ambas heredan de *EstrategiaSeleccionNodo*, que define la interfaz común para decidir qué nodo debe expandirse antes. Así, el algoritmo principal no cambia y solo se sustituye la estrategia de selección.

Tablas de resultados

I. Algoritmo constructivo voraz

PROBLEMA	n	K	m	z	S	CPU
max_div_15_2	15	2	2	11.859216	{9, 7}	0.003
max_div_15_2	15	2	3	25.726200	{9, 7, 4}	0.004
max_div_15_2	15	2	4	48.413931	{9, 7, 4, 11}	0.004
max_div_15_2	15	2	5	73.561899	{9, 7, 4, 11, 2}	0.004
max_div_15_3	15	3	2	13.273240	{12, 9}	0.004
max_div_15_3	15	3	3	30.324088	{12, 9, 5}	0.010
max_div_15_3	15	3	4	59.763759	{12, 9, 5, 11}	0.005
max_div_15_3	15	3	5	94.748719	{12, 9, 5, 11, 14}	0.005
max_div_20_2	20	2	2	8.510329	{18, 19}	0.004
max_div_20_2	20	2	3	21.996086	{18, 19, 9}	0.004
max_div_20_2	20	2	4	39.568229	{18, 19, 9, 3}	0.005
max_div_20_2	20	2	5	61.239291	{18, 19, 9, 3, 13}	0.005
max_div_20_3	20	3	2	11.800309	{13, 14}	0.004
max_div_20_3	20	3	3	30.872658	{13, 14, 8}	0.004
max_div_20_3	20	3	4	56.534726	{13, 14, 8, 3}	0.005

max_div_20_3	20	3	5	92.829743	{13, 14, 8, 3, 17}	0.006
max_div_30_2	30	2	2	11.657144	{9, 28}	0.004
max_div_30_2	30	2	3	28.944300	{9, 28, 2}	0.005
max_div_30_2	30	2	4	52.771171	{9, 28, 2, 11}	0.005
max_div_30_2	30	2	5	80.910241	{9, 28, 2, 11, 13}	0.006
max_div_30_3	30	3	2	13.073737	{17, 7}	0.004
max_div_30_3	30	3	3	33.842259	{17, 7, 24}	0.005
max_div_30_3	30	3	4	63.518414	{17, 7, 24, 14}	0.006
max_div_30_3	30	3	5	99.508836	{17, 7, 24, 14, 15}	0.006

II. Ramificación y Poda (estrategia menor cota superior)

PROBLEMA	n	K	m	z	S	CPU	nº de nodos generados
max_div_15_2	15	2	2	11.859216	{9, 7}	0.017	1
max_div_15_2	15	2	3	27.372700	{1, 7, 9}	0.090	131
max_div_15_2	15	2	4	49.826781	{1, 6, 7, 9}	0.188	269
max_div_15_2	15	2	5	79.129530	{1, 4, 6, 7, 9}	0.316	510
max_div_15_3	15	3	2	13.273240	{12, 9}	0.016	1
max_div_15_3	15	3	3	31.868526	{5, 7, 12}	0.081	99
max_div_15_3	15	3	4	59.763759	{12, 9, 5, 11}	0.119	140
max_div_15_3	15	3	5	96.085832	{4, 5, 9, 12, 14}	0.194	258
max_div_20_2	20	2	2	8.510329	{18, 19}	0.021	1
max_div_20_2	20	2	3	21.996086	{9, 18, 19}	0.198	105
max_div_20_2	20	2	4	40.002258	{2, 3, 9, 19}	0.447	404
max_div_20_2	20	2	5	63.651679	{2, 9, 14, 18, 19}	0.940	1099

max_div_20_3	20	3	2	11.800309	{13, 14}	0.021	1
max_div_20_3	20	3	3	30.872658	{13, 14, 8}	0.185	100
max_div_20_3	20	3	4	56.690310	{3, 13, 14, 17}	0.313	260
max_div_20_3	20	3	5	92.829743	{3, 8, 13, 14, 17}	0.330	242
max_div_30_2	30	2	2	11.657144	{9, 28}	0.036	1
max_div_30_2	30	2	3	28.944300	{2, 9, 28}	0.430	97
max_div_30_2	30	2	4	52.771171	{9, 28, 2, 11}	1.167	303
max_div_30_2	30	2	5	80.910241	{2, 9, 11, 13, 28}	3.664	1386
max_div_30_3	30	3	2	13.073737	{17, 7}	0.034	1
max_div_30_3	30	3	3	34.290529	{6, 17, 24}	0.688	184
max_div_30_3	30	3	4	63.701960	{6, 14, 17, 24}	1.419	396
max_div_30_3	30	3	5	99.592041	{6, 14, 15, 17, 24}	3.455	1065

III. Ramificación y Poda (estrategia mayor cota superior)

PROBLEMA	n	K	m	z	S	CPU	n° de nodos generados
max_div_15_2	15	2	2	11.859216	{9, 7}	0.030	1
max_div_15_2	15	2	3	27.372700	{1, 7, 9}	0.085	89
max_div_15_2	15	2	4	49.826781	{1, 6, 7, 9}	0.170	235
max_div_15_2	15	2	5	79.129530	{1, 4, 6, 7, 9}	0.277	375
max_div_15_3	15	3	2	13.273240	{12, 9}	0.020	1
max_div_15_3	15	3	3	31.868526	{5, 7, 12}	0.073	73
max_div_15_3	15	3	4	59.763759	{12, 9, 5, 11}	0.119	140
max_div_15_3	15	3	5	96.085832	{4, 5, 9, 12, 14}	0.180	224
max_div_20_2	20	2	2	8.510329	{18, 19}	0.023	1

max_div_20_2	20	2	3	21.996086	{9, 18, 19}	0.159	105
max_div_20_2	20	2	4	40.002258	{2, 3, 9, 19}	0.425	364
max_div_20_2	20	2	5	63.651679	{2, 9, 14, 18, 19}	0.720	763
max_div_20_3	20	3	2	11.800309	{13, 14}	0.021	1
max_div_20_3	20	3	3	30.872658	{13, 14, 8}	0.185	100
max_div_20_3	20	3	4	56.690310	{3, 13, 14, 17}	0.288	244
max_div_20_3	20	3	5	92.829743	{3, 8, 13, 14, 17}	0.330	242
max_div_30_2	30	2	2	11.657144	{9, 28}	0.036	1
max_div_30_2	30	2	3	28.944300	{2, 9, 28}	0.445	97
max_div_30_2	30	2	4	52.771171	{9, 28, 2, 11}	1.182	303
max_div_30_2	30	2	5	80.910241	{2, 9, 11, 13, 28}	3.484	1386
max_div_30_3	30	3	2	13.073737	{17, 7}	0.034	1
max_div_30_3	30	3	3	34.290529	{6, 17, 24}	0.471	123
max_div_30_3	30	3	4	63.701960	{6, 14, 17, 24}	1.324	352
max_div_30_3	30	3	5	99.592041	{6, 14, 15, 17, 24}	3.630	1052

Conclusiones

Los resultados muestran que el algoritmo voraz es el más rápido, con tiempos muy reducidos en todas las instancias, pero no siempre alcanza el mejor valor objetivo.

La ramificación y poda obtiene mejores soluciones, y al comprobar las estrategias, en ambas variantes alcanzan el mismo valor óptimo final en las 24 combinaciones evaluadas. En concreto, para $m = 2$ el valor obtenido por el voraz coincide con el de ramificación y poda en todas las instancias, mientras que para $m > 2$ la rama exacta mejora la calidad de



la solución frente al constructivo voraz. Esto confirma que el voraz funciona bien como aproximación inicial, pero no garantiza optimalidad.

Entre las dos estrategias de ramificación, la que prioriza la mayor cota superior suele ser ligeramente mejor que la de menor cota superior en tiempo y en número de nodos generados. En general, la de cota superior explora menos nodos y tarda algo menos, aunque la diferencia no es enorme en las instancias usadas. Aun así, la ventaja es consistente y hace más recomendable esta estrategia para la implementación final.

En general el programa confirma que el voraz es útil como solución rápida de referencia y como cota inferior inicial, mientras que la ramificación y poda proporciona soluciones óptimas con un coste mayor pero todavía asumible para las instancias de la práctica.